

EXPOSÉ
Leonhard Lüdeke

Proove of Concept für die Implementierung des A2A Protokolls in die KI-Infrastruktur von Willhlem.tel

FAKULTÄT INFORMATIK UND DIGITALE GESELLSCHAFT

Faculty of Computer Science and Digital Society

Betreuung durch: ???
Eingereicht am: 27. März 2026

HOCHSCHULE FÜR ANGEWANDTE
WISSENSCHAFTEN HAMBURG
Hamburg University of Applied Sciences

Inhaltsverzeichnis

1	Problemstellung und Relevanz	2
2	Forschungsstand	3
2.1	Agent-Kommunikationsprotokolle und A2A	3
2.2	Python A2A und Implementierungsbibliotheken	4
2.3	Integration mit Multi-Agent-Architekturen	4
3	Zielsetzung und Forschungsfragen	5
4	Methodik und geplanter Aufbau	7
4.1	Vorgehensweise / Methodik	7
4.2	Vorläufige Gliederung der Arbeit	8
5	Erwarteter Beitrag der Arbeit	8
6	Vorläufige Literatúrauswahl (Auszug)	9
	Literatur	10

Keywords: Agent 2 Agent Protokoll, KI-Agenten, Multi-Agent-Systeme

1 Problemstellung und Relevanz

Mit der Verbreitung agentischer KI-Systeme entstehen zunehmend Multi-Agent-Umgebungen, in denen spezialisierte KI-Agenten gemeinsam komplexe Aufgaben bearbeiten. Ohne standardisierte Kommunikationsprotokolle führen proprietäre Integrationen zu hoher Kopplung, eingeschränkter Wiederverwendbarkeit und mangelnder Interoperabilität zwischen Agenten, Frameworks und Infrastrukturen.[3, 7, 2, 4]

Der Agent2Agent-Standard (A2A) adressiert diese Problematik, indem er ein offenes, protokollbasiertes Kommunikationsmodell für KI-Agenten definiert. A2A ist als plattform- und frameworkunabhängiger Messaging-Layer konzipiert, der auf etablierten Web-Standards wie HTTP, JSON-RPC und Server-Sent Events (SSE) aufsetzt und so eine lose gekoppelte, interoperable Kommunikation sowie Peer-to-Peer-Taskdelegation zwischen Agenten unterschiedlicher Anbieter ermöglicht.[7, 3, 1, 12, 6]

Kernbausteine des Protokolls sind Agent Cards (Selbstbeschreibung von Agenten mit Fähigkeiten, Transportprotokollen und Endpunkten), Tasks mit definiertem Lebenszyklus, Messages als Kommunikationseinheiten sowie Artifacts als greifbare Ergebnisse.[1, 3, 12, 5] Öffentlich verfügbare technische Beschreibungen und Bibliotheken positionieren A2A als offenen Standard, mit dem Agenten Fähigkeiten über Agent Cards bekannt machen, Tasks austauschen und Ergebnisse als Artifacts bereitstellen können.[1, 5, 3]

Für ein Unternehmen wie wilhelm.tel, das bereits containerisierte KI-Dienste und LLM-basierte Agenten in einer Hybrid-Cloud-Umgebung einsetzt, eröffnet A2A die Möglichkeit, bestehende Agenten-Workloads von proprietären Integrationen auf einen standardisierten Kommunikationslayer zu migrieren, ohne die Kernlogik der Agenten vollständig neu zu implementieren. Die geplante Bachelorarbeit adressiert damit eine aktuelle Herausforderung der industriellen Einführung agentischer KI und fügt sich in die Diskussion um agentische Softwarearchitekturen und Agenteninfrastruktur ein.[2, 4, 3, 7]

2 Forschungsstand

2.1 Agent-Kommunikationsprotokolle und A2A

Die aktuelle Forschung zu Agentenkommunikationsprotokollen zeigt, dass mit der zunehmenden Verbreitung LLM-basierter, autonomer Agenten die Anforderungen an Interoperabilität, Sicherheit und Skalierbarkeit deutlich steigen.[3, 7, 4, 2] Ehtesham et al. vergleichen in ihrem Survey Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent2Agent (A2A) und Agent Network Protocol (ANP) und zeigen, dass A2A primär auf Peer-to-Peer-Taskdelegation und kollaborative Aufgabenerfüllung zwischen autonomen Agenten fokussiert, während MCP stärker auf sicheren Toolzugriff und strukturierte Kontextanreicherung abzielt und ACP eine REST-konforme, MIME- und session-fähige Nachrichtenebene bereitstellt.[7, 3]

Ray gibt in seiner Übersicht zu A2A einen systematischen Überblick über Konzept, Stand der Technik, Herausforderungen und zukünftige Forschungsrichtungen. Duan und Lu analysieren den Einsatz von A2A im Kontext von Edge Computing und zeigen, dass das Protokoll prinzipiell geeignet ist, aber unter Randbedingungen wie knappen Ressourcen, Latenzanforderungen und heterogenen Netzen weiter untersucht werden muss.[12, 6]

Öffentlich zugängliche Spezifikationen und Projektressourcen stellen A2A als offenen Standard vor, der auf HTTP, JSON-RPC 2.0 und SSE aufbaut und einen klar definierten Task-Lebenszyklus, Agent Cards, Messages und Artifacts spezifiziert. Die Open-Source-Initiativen rund um A2A konkretisieren diesen Standard und diskutieren Erweiterungen wie Agent-Registries, dynamische Agent Cards und erweiterte Task-Management-Funktionalität.[1, 5, 3, 8]

2.2 Python A2A und Implementierungsbibliotheken

Für die Implementierung von A2A in Python steht mit *Python A2A / a2a-python* ein umfassendes, produktionsorientiertes SDK zur Verfügung, das die Spezifikation des A2A-Protokolls weitgehend vollständig abbildet. Die Bibliothek stellt u. a. folgende Funktionalitäten bereit:[5]

- Implementierung der offiziellen A2A-Spezifikation einschließlich Agent Cards, Tasks, Messages und Artifacts,
- Unterstützung für Agentenregistrierung und -entdeckung über Agent Cards,
- eingebaute Unterstützung für die Nutzung von MCP zur Standardisierung von Toolzugriff und Kontextanreicherung,
- Integrationsmöglichkeiten mit gängigen LLM-Frameworks und -Anbietern,
- produktionsorientierte Features wie robuste Fehlerbehandlung und eine auf Interoperabilität ausgelegte API.[5, 1]

Das zugrundeliegende Open-Source-Projekt A2A pflegt neben der Spezifikation eine Roadmap für SDKs in verschiedenen Programmiersprachen und diskutiert aktiv Erweiterungen wie Task-Listenendpunkte, Agent-Registrare und dynamische Agent Cards. Diese Community-Entwicklung macht A2A und a2a-python zu geeigneten Kandidaten für praxisorientierte Experimente in Unternehmenskontexten.[1, 3, 5]

2.3 Integration mit Multi-Agent-Architekturen

Die Literatur zu Multi-Agent-Systemen und agentischer KI betont, dass standardisierte Kommunikationsschichten eine Voraussetzung für skalierbare, sicher betreibbare Agentennetze in Unternehmen sind.[2, 4, 10, 11] Ehtesham et al. sowie weitere Arbeiten zu agentischen Architekturen zeigen, dass Protokolle wie MCP, ACP und A2A komplementäre Rollen einnehmen: MCP für Toolzugriffe, ACP für HTTP-basierte, session-aware Nachrichtenflüsse und A2A für kollaborative Taskausführung zwischen autonomen Agenten.[7, 3, 13]

Li und Xie argumentieren, dass A2A und MCP zusammen ein starkes Fundament für skalierbare Agentensysteme bilden, gleichzeitig aber neue Herausforderungen in Bezug auf

Sicherheit, semantische Interoperabilität und Governance entstehen.[8] Liao et al. demonstrieren mit AgentMaster[9] ein multi-protokollfähiges Agentenframework, das A2A mit MCP kombiniert, um flexible, domänenspezifische Multi-Agent-Systeme für multimodale Informationssuche und Analyse zu realisieren.[9] Srivastava et al. zeigen mit EffGen, wie Multi-Agent-Systeme und Protokolle wie A2A, MCP und ACP in einem agentischen Framework für Small Language Models zusammengeführt werden können.[13]

Alenezi ordnet diese Entwicklungen in eine größere Architekturperspektive ein und beschreibt den Übergang von reinen Prompt-Response-Systemen hin zu zielgerichteten, agentischen Architekturen mit standardisierten Agent Loops, Protokollen, Registries und auditierbaren Kontrollmechanismen.[2] Insgesamt lässt sich festhalten, dass A2A in der aktuellen Literatur als wichtiger Baustein für interoperable Multi-Agent-Systeme identifiziert wird, dass aber praktische Fallstudien zur Einbettung in konkrete Unternehmensinfrastrukturen und Hybrid-Cloud-Umgebungen erst im Entstehen begriffen sind.[3, 4, 6]

3 Zielsetzung und Forschungsfragen

Die Bachelorarbeit verfolgt das Ziel, einen lauffähigen Proof-of-Concept zu entwerfen und zu implementieren, in dem drei spezialisierte KI-Agenten und ein Orchestrator über das Agent2Agent-Protokoll (A2A) miteinander kommunizieren und in die bestehende Docker- und Kubernetes-Infrastruktur von wilhelm.tel integrierbar sind. Dabei soll explizit auf die Python-Bibliothek *a2a-python* zurückgegriffen werden, um eine protokollkonforme Implementierung zu gewährleisten.[5, 1]

Der Fokus liegt bewusst auf der funktionalen Umsetzbarkeit, der Architektur und der protokolltreuen Nutzung von A2A. Detaillierte Performanzmessungen, Skalierungsexperimente oder umfassende Sicherheitsanalysen werden lediglich qualitativ, basierend auf Beobachtungen (z. B. Logs, einfache Metriken), diskutiert.

Aus dieser Zielsetzung ergeben sich folgende, in einem Bachelorarbeitsumfang realistisch bearbeitbare Forschungsfragen:

1. **Wie kann die Python-Bibliothek a2a-python in eine bestehende Agentenarchitektur integriert werden, um drei spezialisierte Agenten und einen Orchestrator über das A2A-Protokoll miteinander zu verbinden?**

Diese Frage adressiert die konkrete Nutzung der a2a-python-API, die Modellierung von Agent Cards, Tasks und Messages sowie die Implementierung einer protokolltreuen Client-Server-Interaktion zwischen Agenten in Python.[5, 1]

2. **Welche Anpassungen sind erforderlich, um die A2A-basierten Agenten und den Orchestrator in Form von Docker-Containern auf Kubernetes zu deployen und über standardisierte Endpunkte ansprechbar zu machen?**

Im Vordergrund steht hier die Abbildung der A2A-Endpunkte (z. B. HTTP-/JSON-RPC-Schnittstellen) auf Kubernetes-Services und -Deployments sowie die Integration in bestehende CI/CD- und Monitoring-Strukturen von wilhelm.tel.[4, 11]

3. **Inwieweit lässt sich ein exemplarischer End-to-End-Workflow (z. B. Recherche → Analyse → Reporting) mittels A2A orchestrieren, sodass der Orchestrator Aufgaben sinnvoll delegiert und Ergebnisse aggregiert?**

Diese Frage untersucht, wie der in der Spezifikation definierte Task-Lebenszyklus (inklusive Zwischenzuständen und Artifacts) genutzt werden kann, um einen geschlossenen fachlichen Ablauf über mehrere spezialisierte Agenten hinweg zu realisieren.[1, 3]

- Optional* **Welche architektonischen Chancen und Grenzen ergeben sich aus der Nutzung von A2A und a2a-python in einer Hybrid-Cloud-Umgebung wie bei wilhelm.tel?**

Hier stehen eine reflektierende Betrachtung der Interoperabilitätsgewinne, der Integrationsaufwände sowie möglicher Limitierungen (z. B. Sicherheitsaspekte, Observability, Debugging-Komplexität) auf Basis der Literatur und der PoC-Erfahrungen im Fokus.[8, 2, 6, 14, 4]

Der Umfang dieser Arbeit bleibt damit auf einen lauffähigen PoC mit dokumentierter Architektur, Implementierung und funktionaler Evaluation beschränkt. Weitergehende

Themen wie umfassende Benchmarks, großskalige Experimente oder tiefgehende Sicherheitsanalysen sind nicht teil dieser Arbeit und werden explizit als Ausblick für zukünftige Arbeiten gekennzeichnet.[4, 10]

4 Methodik und geplanter Aufbau

4.1 Vorgehensweise / Methodik

Die Arbeit folgt einem empirisch-technischen Vorgehen mit Schwerpunkt auf Softwareentwicklung und Architektur. Geplant sind insbesondere folgende Schritte:

1. **Anforderungsanalyse:** Erhebung der relevanten Rahmenbedingungen der bestehenden Infrastruktur von wilhelm.tel (z. B. Kubernetes-Cluster, vorhandene LLM-Dienste, CI/CD-Vorgaben) sowie Ableitung funktionaler und einfacher nicht-funktionaler Anforderungen an den PoC (z. B. Anzahl der Agenten, Kommunikationsmuster, grundlegende Sicherheitsanforderungen auf Protokollebene).[2, 4]
2. **Architektur- und Schnittstellendesign:** Entwurf einer Referenzarchitektur mit drei spezialisierten Agenten (z. B. Research-Agent, Analyse-Agent, Reporting-Agent) und einem Orchestrator. Definition der Agent Cards, Fähigkeiten, Endpunkte und Kommunikationsflüsse gemäß A2A-Spezifikation.[1, 5]
3. **Implementierung mit a2a-python:** Entwicklung/Anpassung der Agenten und des Orchestrators auf Basis der a2a-python-Bibliothek. Die bisherigen Agenten sind mit dem Langchain Framework entwickelt und besitzen eine anbindung an vorhandene LLM-Dienste in der wilhelm.tel-Umgebung. Das a2a-python Framework bietet eine Möglichkeit LangChain componenten zu A2A Servern zu convertieren. Die Entwicklung des Orchestrator Agenten findet mittels der des Mitgelieferten AgentNetwork System statt.
4. **Containerisierung und Deployment:** Erstellung von Docker-Images für die Agenten und den Orchestrator, Definition der notwendigen Kubernetes-Deployments und -Services sowie Integration in bestehende Deployment-Pipelines von wilhelm.tel.[4, 11]

5. **Funktionale Tests und Demonstration:** Durchführung von Tests zur Überprüfung der A2A-Kommunikation, Aufbau eines Demo-Szenarios (z. B. Nutzeranfrage, die durch alle drei Agenten verarbeitet wird), Log-Analysen und einfache Metriken zur Verifikation der funktionalen Korrektheit.
6. **Reflexion und Ausblick:** Bewertung der Ergebnisse im Hinblick auf Interoperabilität, Erweiterbarkeit, Betreibbarkeit und Anschlussfähigkeit an andere Protokolle (z. B. MCP, ACP, ANP) oder Frameworks. Ableitung von Empfehlungen und Identifikation von Forschungs- und Verbesserungsbedarf.[7, 8, 4, 14]

4.2 Vorläufige Gliederung der Arbeit

1. Einleitung
2. Theoretische Grundlagen (agentische KI, Multi-Agent-Systeme, Agentenkommunikationsprotokolle, A2A-Spezifikation, a2a-python, Grundlagen von Kubernetes/Docker)
3. Anforderungsanalyse und Architekturentwurf für den A2A-basierten PoC bei wilhelm.tel
4. Implementierung des Proof-of-Concept mit a2a-python
5. Deployment, Tests und funktionale Evaluation
6. Diskussion, Limitationen und Ausblick

5 Erwarteter Beitrag der Arbeit

Die Arbeit leistet einen praxisorientierten Beitrag zur aktuellen Diskussion um agentische KI und Protokollstandardisierung, indem sie:

- A2A und a2a-python exemplarisch in einer unternehmensnahen Hybrid-Cloud-Umgebung einsetzt und dokumentiert,
- einen konkreten Referenzaufbau für drei spezialisierte Agenten und einen Orchestrator liefert,
- Erfahrungen, Chancen und Grenzen bei der Einführung von A2A im Unternehmenskontext reflektiert und

- Anknüpfungspunkte für weiterführende Arbeiten (z. B. Performanzanalysen, Sicherheitsarchitekturen, Integration mit MCP und ACP) identifiziert.[9, 13, 8, 4, 3]

Damit bleibt der Umfang klar im Rahmen einer Bachelorarbeit, adressiert jedoch ein aktuelles und forschungsnahes Thema mit hoher praktischer Relevanz.

6 Vorläufige Literatúrauswahl (Auszug)

Die folgenden Quellen bilden den Kern der theoretischen Fundierung der Arbeit:

1. Ehtesham, A.; Singh, A.; Gupta, G. K.; Kumar, S. (2025): *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. arXiv:2505.02279.[7]
2. Aniket P. Kakde; Karan M. Bhoyar; Muhammad Aiman Shad; Prof. Sudesh A. Bachwani (2025): *Advancing Agentic AI through Communication Protocols*. Int. J. Sci. Res. Sci. Technol.[3]
3. Srivastava, G. et al. (2026): *EffGen: Enabling Small Language Models as Capable Autonomous Agents*. arXiv:2602.00887.[13]
4. Alenezi, M. (2026): *From Prompt-Response to Goal-Directed Systems: The Evolution of Agentic AI Software Architecture*. arXiv:2602.10479.[2]
5. Marro, S. et al. (2024): *A Scalable Communication Protocol for Networks of Large Language Models (Agora)*. arXiv:2410.11905.[10]
6. Olaru, A.; Pricope, M. (2023): *Multi-Modal Decentralized Interaction in Multi-Entity Systems*. Sensors 23(6):3139.[11]
7. Stappen, L. et al. (2026): *Agent2Agent Threats in Safety-Critical LLM Assistants: A Human-Centric Taxonomy*. arXiv:2602.05877.[14]
8. Chan, A. et al. (2025): *Infrastructure for AI Agents*. arXiv:2501.10114.[4]
9. Ray, P. P. (2025): *A Review on Agent-to-Agent Protocol: Concept, State-of-the-art, Challenges and Future Directions*. TechRxiv.[12]

10. Duan, Q.; Lu, Z. (2025): *Agent Communications toward Agentic AI at Edge – A Case Study of the Agent2Agent Protocol*. arXiv:2508.15819.[6]
11. Desai, M. (2025): *A comprehensive Python library for Google’s Agent-to-Agent (A2A) protocol*. GitHub/python-a2a.[5]
12. Li, Q.; Xie, Y. (2025): *From Glue-Code to Protocols: A Critical Analysis of A2A and MCP Integration for Scalable Agent Systems*. arXiv:2505.03864.[8]
13. Liao, C. C.; Liao, D.; Gadiraju, S. S. (2025): *AgentMaster: A Multi-Agent Conversational Framework Using A2A and MCP Protocols for Multimodal Information Retrieval and Analysis*. arXiv:2507.21105.[9]

Weitere wissenschaftliche Publikationen zu Multi-Agent-Systemen, Protokollarchitekturen und agentischer KI werden im Verlauf der Arbeit zielgerichtet ergänzt.

Literatur

- [1] : *A2A Google*. – URL <https://a2a-protocol.org/latest/>
- [2] ALENEZI, Mamdouh: *From Prompt-Response to Goal-Directed Systems: The Evolution of Agentic AI Software Architecture*. – URL <http://arxiv.org/abs/2602.10479>. – Zugriffsdatum: 2026-03-20
- [3] ANIKET P. KAKDE ; KARAN M. BHOYAR ; MUHAMMAD AIMAN SHAD ; PROF. SUDESH A. BACHWANI: *Advancing Agentic AI through Communication Protocols*. 12, Nr. 5, S. 299–308. – URL <https://ijsrst.com/index.php/home/article/view/IJSRST25125127>. – Zugriffsdatum: 2026-03-27. – ISSN 2395-602X, 2395-6011
- [4] CHAN, Alan ; WEI, Kevin ; HUANG, Sihao ; RAJKUMAR, Nitarshan ; PERRIER, Elija ; LAZAR, Seth ; HADFIELD, Gillian K. ; ANDERLJUNG, Markus: *Infrastructure for AI Agents*. – URL <https://arxiv.org/abs/2501.10114>. – Zugriffsdatum: 2026-03-27. – Version Number: 3
- [5] DESAI, Manoj: *A comprehensive Python library for Google’s Agent-to-Agent (A2A) protocol*. – URL <https://github.com/themanojdesai/python-a2a>

- [6] DUAN, Qiang ; LU, Zhihui: *Agent Communications toward Agentic AI at Edge – A Case Study of the Agent2Agent Protocol*. – URL <https://arxiv.org/abs/2508.15819>. – Zugriffsdatum: 2026-03-19. – Version Number: 1
- [7] EHTESHAM, Abul ; SINGH, Aditi ; GUPTA, Gaurav K. ; KUMAR, Saket: *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. – URL <http://arxiv.org/abs/2505.02279>. – Zugriffsdatum: 2025-11-01
- [8] LI, Qiaomu ; XIE, Ying: *From Glue-Code to Protocols: A Critical Analysis of A2A and MCP Integration for Scalable Agent Systems*. – URL <https://arxiv.org/abs/2505.03864>. – Zugriffsdatum: 2026-03-19. – Version Number: 1
- [9] LIAO, Callie C. ; LIAO, Duoduo ; GADIRAJU, Sai S.: *AgentMaster: A Multi-Agent Conversational Framework Using A2A and MCP Protocols for Multimodal Information Retrieval and Analysis*. – URL <http://arxiv.org/abs/2507.21105>. – Zugriffsdatum: 2026-03-19
- [10] MARRO, Samuele ; LA MALFA, Emanuele ; WRIGHT, Jesse ; LI, Guohao ; SHADBOLT, Nigel ; WOOLDRIDGE, Michael ; TORR, Philip: *A Scalable Communication Protocol for Networks of Large Language Models*. – URL <https://arxiv.org/abs/2410.11905>. – Zugriffsdatum: 2026-03-27. – Version Number: 1
- [11] OLARU, Andrei ; PRICOPE, Monica: *Multi-Modal Decentralized Interaction in Multi-Entity Systems*. 23, Nr. 6, S. 3139. – URL <https://www.mdpi.com/1424-8220/23/6/3139>. – Zugriffsdatum: 2026-03-27. – ISSN 1424-8220
- [12] RAY, Partha P.: *A Review on Agent-to-Agent Protocol: Concept, State-of-the-art, Challenges and Future Directions*. – URL <https://www.techrxiv.org/users/913189/articles/1289879-a-review-on-agent-to-agent-protocol-concept-state-of-the-art-challenges-and-future-directions?commit=89695e7bd539757442ec066a8f6ec74f31c0dec1>. – Zugriffsdatum: 2026-03-19
- [13] SRIVASTAVA, Gaurav ; HUSSAIN, Aafiya ; WANG, Chi ; LIN, Yingyan C. ; WANG, Xuan: *EffGen: Enabling Small Language Models as Capable Autonomous Agents*. – URL <http://arxiv.org/abs/2602.00887>. – Zugriffsdatum: 2026-03-19

- [14] STAPPEN, Lukas ; TURAN, Ahmet E. ; HAGERER, Johann ; GROH, Georg:
Agent2Agent Threats in Safety-Critical LLM Assistants: A Human-Centric Taxo-
nomy. – URL <https://arxiv.org/abs/2602.05877>. – Zugriffsdatum:
2026-03-27. – Version Number: 1